Proper Notes Format

◆ @RestController AND @RequestMapping("/email")

```
@RestController
@RequestMapping("/email")
public class EmailController {
```

This is the beginning of your controller class.

Spring reads these annotations when the application starts.

These annotations tell Spring:

> "This class is responsible for handling web requests."

What is @RestController?

```
@RestController
```

This is an annotation.

Annotations are special instructions given to Spring.

Think of annotations as sticky notes attached to your code.

Example:

```
@RestController
```

is like attaching a note saying:

> "Spring, this class is a REST API controller."

What does REST API mean?

A REST API is simply a program that receives requests and returns responses.

Example:

Client sends:

```
POST /email/analyze
```

Server returns:

```
{
  "result":"SPAM",
  "confidence":1.0
}
```

The controller is the class that receives the request.

What happens when Spring starts?

When you run:

```
@SpringBootApplication
public class SmartEmailThreatDetectionSystemApplication {

    public static void main(String[] args) {
        SpringApplication.run(...);
    }

}
```

Spring begins scanning your project.

It looks through all classes.

When it finds:

```
@RestController
```

Spring says:

> "This class handles web requests."

Then Spring registers the controller internally.

What if @RestController is removed?

Suppose you write:

```
public class EmailController {
}
```

without:

```
@RestController
```

Spring will ignore this class.

No URL will work.

Example:

```
POST /email/analyze
```

will return:

```
404 Not Found
```

because Spring never registered the controller.

Why do we need @RestController?

Without it:

```
EmailController
```

is just a normal Java class.

Spring doesn't know it should handle requests.

With it:

```
@RestController
```

Spring knows:

> "Use this class for web requests."

What does RESTController actually combine?

Internally:

```
@RestController
```

is equivalent to:

```
@Controller
@ResponseBody
```

combined together.

@Controller

```
@Controller
```

means:

> "This class handles web requests."

@ResponseBody

```
@ResponseBody
```

means:

> "Return data directly as JSON."

Example:

```
return new EmailResponseDTO("SPAM",1.0);
```

becomes:

```
{
  "result":"SPAM",
```

```
  "confidence":1.0
}
```

automatically.

Because of:

```
@RestController
```

you don't need to manually convert objects into JSON.

Spring does it for you.

◆ @RequestMapping("/email")

```
@RequestMapping("/email")
```

This annotation defines the base URL for the controller.

What is a URL?

Example:

```
http://localhost:8080/email/analyze
```

Break it down:

```
http://localhost:8080
```

Server address.

```
/email/analyze
```

Path.

Spring uses annotations to decide which path belongs to which controller.

What does @RequestMapping("/email") do?

```
@RequestMapping("/email")
```

tells Spring:

> "Every method inside this controller starts with /email."

Think of it as a prefix.

Example:

```
@RequestMapping("/email")
public class EmailController {
}
```

Now every method inside this class automatically starts with:

```
/email
```

Method-Level Mapping

Suppose you have:

```
@PostMapping("/analyze")
```

Spring combines:

```
/email
```

and

```
/analyze
```

to form:

```
/email/analyze
```

Visual Flow

```
@RequestMapping("/email")
        +
@PostMapping("/analyze")
        ↓
/email/analyze
```

Another Example

Suppose you add:

```
@GetMapping("/history")
```

Spring combines:

```
/email
```

and

```
/history
```

Result:

```
/email/history
```

Why use a base URL?

Imagine you have 20 email-related methods.

Without a base URL:

```
@PostMapping("/email/analyze")
@GetMapping("/email/history")
@GetMapping("/email/report")
@PostMapping("/email/delete")
```

You keep repeating:

```
/email
```

again and again.

With:

```
@RequestMapping("/email")
```

you only write it once.

Methods become:

```
@PostMapping("/analyze")
@GetMapping("/history")
@GetMapping("/report")
@PostMapping("/delete")
```

Cleaner and easier to maintain.

◆ EmailService OBJECT

```
private final EmailService emailService;
```

This line confuses many beginners.

Let's break it apart.

EmailService

```
EmailService
```

is a class.

Example:

```
public class EmailService {
}
```

Just like:

```
String
Scanner
ArrayList
```

are classes.

---

emailService

```
emailService
```

is a variable.

It stores a reference to an object.

Example:

```
String name;
```

Here:

```
String
```

is the class.

```
name
```

is the variable.

Similarly:

```
EmailService emailService;
```

means:

```
EmailService
```

Class.

```
emailService
```

Variable.

What object is stored inside?

Eventually Spring creates:

```
new EmailService()
```

and stores it inside:

```
emailService
```

Variable.

Think of:

```
emailService
```

as a remote control.

The actual object exists elsewhere in memory.

What does private mean?

```
private final EmailService emailService;
```

private means:

> "Only this class can use this variable."

Outside classes cannot access it directly.

What does final mean?

```
final
```

means:

> "After assigning a value once, it cannot be changed."

Example:

```
final String name = "John";
```

This is allowed.

But:

```
name = "David";
```

causes an error.

Similarly:

```
private final EmailService emailService;
```

means:

Once Spring assigns the EmailService object, it cannot be replaced.

Why is final useful?

It makes the code safer.

Nobody can accidentally replace:

```
emailService
```

with another object.

Constructor Injection

```
public EmailController(EmailService emailService) {
    this.emailService = emailService;
}
```

Spring automatically calls this constructor.

Suppose Spring creates:

```
EmailService service =
    new EmailService();
```

Then Spring does:

```
new EmailController(service);
```

Inside constructor:

```
this.emailService = emailService;
```

means:

Left side:

```
this.emailService
```

Controller variable.

Right side:

```
emailService
```

Parameter received from Spring.

After assignment:

```
Controller.emailService
        ↓
```

```
EmailService Object
```

Now the controller can use:

```
emailService.analyze(...)
```

Why Dependency Injection?

Instead of controller creating objects itself:

```
EmailService service =
    new EmailService();
```

Spring creates and manages them.

Benefits:

- Less code
- Easier testing
- Better maintenance
- Loose coupling

Simple Analogy

Imagine a restaurant.

```
Customer
   ↓
Waiter
   ↓
Chef
```

Customer:

```
User
```

Waiter:

```
EmailController
```

Chef:

```
EmailService
```

The waiter does not cook food.

The chef cooks food.

Similarly:

Controller receives requests.

Service performs business logic.

In Short

- `@RestController` → Registers this class as a REST API controller.
- `@RequestMapping("/email")` → Base URL for all methods.
- `EmailService` → Class.
- `emailService` → Variable referencing an EmailService object.
- `private` → Only accessible inside the controller.
- `final` → Cannot be reassigned.
- Constructor Injection → Spring automatically provides the EmailService object.
- `emailService.analyze()` → Calls a method inside EmailService.

Next section starts with:

◆ What Spring Boot Actually Does Behind the Scenes (Step 1 → Step 10 Request Flow)

This is where the complete request journey starts:
Swagger → Tomcat → Spring Routing → Controller → Service → Flask → ML Model → Response.